

ABOUALOU Jihan
ALILECHE Samy
BENJAMIN BENEDICT
LE BRETON Nicolas
PERONI Dylan



IUT de Vélizy-Rambouillet
CAMPUS DE VÉLIZY-VILLACOUBLAY

SAE01

Cryptographie

Sommaire

1. Chiffrement par flux vs Chiffrement par bloc.....	3
1.2 - Schéma de fonctionnement global de ChaCha20.....	3
2. Structure ChaCha20.....	4
2.1 - L'état initial de ChaCha20.....	4
2.2 - Opérateur de base.....	4
Constante Fixe.....	5
Nonce.....	6
Clé.....	6
Compteur.....	6
3. Générateur du keystream.....	7
4. Avantages du ChaCha20.....	7
5. Analyse de sécurité.....	8
6. Structure de l'implémentation & Difficultés	
rencontrées.....	9
6.1 - Structure de l'implémentation.....	9
6.2 - Difficultés rencontrées.....	10
7. Mise en forme sur notre plateforme WEB.....	11
8. Limites et perspectives.....	12
8.1 - Comparaison implémentation manuelle vs bibliothèque.....	12
9. Sources/Bibliographies.....	14

Introduction

Cette SAÉ a pour objectif l'implémentation de l'algorithme de chiffrement **ChaCha20**, conformément aux spécifications de la RFC 8439 (Nir & Langley, 2018). ChaCha20 est un chiffrement par flot moderne conçu par Daniel J. Bernstein, aujourd'hui largement utilisé dans des protocoles sécurisés tels que TLS 1.3 et OpenSSH.

Ce projet vise à comprendre en profondeur le fonctionnement interne d'un algorithme de chiffrement par flot, à en réaliser une implémentation complète en Python sans bibliothèque externe, puis à comparer cette implémentation avec une version utilisant une bibliothèque spécialisée.

L'objectif pédagogique est double :

- maîtriser les principes cryptographiques sous-jacents (ARX, keystream, gestion du compteur),
- intégrer cette solution dans une application web afin de sécuriser les identifiants utilisateurs.

Ce rapport présente d'abord les principes théoriques de ChaCha20, puis la structure de l'implémentation réalisée, avant d'analyser les difficultés rencontrées et les limites de cette approche.

1. Chiffrement par flux vs Chiffrement par bloc

ChaCha20 est un **chiffrement par flux** (ou *stream cipher*).

Un chiffrement par flux génère un flot de clés pseudo-aléatoires (keystream) qui est combiné au message clair, octet par octet, à l'aide d'une opération XOR.

À l'inverse, un chiffrement par bloc (comme AES) traite les données par blocs de taille fixe (par exemple 128 bits), ce qui nécessite parfois des mécanismes supplémentaires (padding, modes opératoires).

ChaCha20 présente plusieurs avantages par rapport aux chiffrements par bloc.

- Il est naturellement adapté aux flux de données continus,
- Il ne nécessite pas d'alignements par blocs,
- Il est très performant sur les processeurs ne disposant pas d'instructions matérielles dédiées à AES.

Pour ces raisons, ChaCha20 est largement utilisé comme alternative à AES dans des protocoles modernes tels que TLS 1.3 ou OpenSSH.

1.2 - Schéma de fonctionnement global de ChaCha20

ChaCha20 fonctionne à partir d'un état interne représenté par une matrice 4 x 4 mots de 32 bits (soit 512 bits au total).

A partir de cet état initial :

1. l'algorithme applique 20 rounds de transformation, basés sur des opérations ARX (Addition, Rotation, XOR) ;
2. l'état final obtenu est additionné mot à mot à l'état initial ;
3. le résultat est converti en 64 octets de keystream ;
4. Ce keystream est combiné au message clair par une opération XOR pour produire le message chiffré (ou inversement pour le déchiffrement).

Le chiffrement et le déchiffrement utilisent exactement le **même algorithme**, seule l'entrée (texte clair ou chiffré) change.

2. Structure ChaCha20

Cette section décrit la structure interne de ChaCha20 et le rôle de chaque composant de son état initial.

2.1 - L'état initial de ChaCha20

L'état initial est composé de 16 mots de 32 bits, organisés comme suit :

Mots de 32 bits chacun.			
Constante	Constante	Constante	Constante
Clé	Clé	Clé	Clé
Clé	Clé	Clé	Clé
Compteur	Nonce	Nonce	Nonce

2.2 - Opérateur de base

L'opération de base est le "quart de tour" (Quarter Round). Il prend quatre nombres (a, b, c, d) et applique une série d'additions, de XOR et de rotations binaires ARX(Add, Rotate, XOR):

1. $a = a + b$; $d = d \text{ XOR } a$; $d = d \lll 16$

2. $c = c + d$; $b = b \text{ XOR } c$; $b = b \lll 12$

3. $a = a + b$; $d = d \text{ XOR } a$; $d = d \lll 8$

4. $c = c + d$; $b = b \text{ XOR } c$; $b = b \lll 7$

Ces opérations sont répétées selon un schéma précis de **rounds sur colonnes** puis de **rounds sur diagonales**, pour un total de 20 rounds (10 doubles rounds).

Constante Fixe

Les constantes sont définies par les quatre mots de 32 bits suivants (en hexadécimal):

1. 0x61707865
2. 0x3320646e
3. 0x79622d32
4. 0x6b206574

Si vous convertissez ces valeurs hexadécimales en caractères ASCII (en tenant compte de l'ordre little-endian), vous obtenez la phrase magique : **"expand 32-byte k"** qui signifie "étendre une clé de 32 octets".

Les constantes ne sont pas là par hasard ; elles remplissent plusieurs fonctions critiques pour la sécurité:

- **Élimination de la linéarité** : Elles garantissent que l'état initial ne contient pas trop de zéros ou de structures répétitives que pourrait exploiter un cryptanalyste.
- **Brise-symétrie** : Elles permettent de s'assurer que chaque mot de l'état réagit différemment lors des opérations mathématiques, empêchant ainsi des relations simples entre les colonnes ou les lignes au début du calcul.
- **Preuve de "Nothing up my sleeve" (rien dans mes manches)** : En utilisant une phrase lisible comme constante, l'inventeur (Daniel J. Bernstein) démontre qu'il n'a pas choisi ces chiffres spécifiquement pour créer une "porte dérobée" (backdoor) cachée dans l'algorithme.

Nonce

Il occupe les 3 derniers mots de l'état initial.

Son rôle :

- Il garantit que deux messages chiffrés avec la même clé produisent des résultats différents;
- Il empêche les attaques par réutilisation de keystream.

Taille : Dans la version IETF (définie dans ce document), le nonce fait **96 bits** (soit 12 octets).

La règle d'or (Usage unique) : Comme son nom l'indique, un nonce ne doit **jamais être réutilisé** avec la même clé.

- Si vous utilisez la même paire (Clé, Nonce) pour chiffrer deux messages différents, un attaquant peut effectuer une opération mathématique simple (XOR) entre les deux textes chiffrés pour annuler le chiffrement et potentiellement retrouver vos messages en clair.

Pas besoin d'être secret : Contrairement à la clé, le nonce n'a pas besoin d'être caché. Il est généralement envoyé en clair en même temps que le message chiffré pour que le destinataire puisse utiliser le même nonce lors du déchiffrement.

Clé

La clé de ChaCha20 est une clé secrète de 256 bits (32 octets), répartie sur 8 mots de 32 bits dans l'état initial.

Compteur

Le compteur est un entier de 32 bits, incrémenté pour chaque bloc de 64 octets généré.

Il permet :

- de chiffrer des messages de taille arbitraire,
- de garantir que chaque bloc utilise un keystream différent.

3. Générateur du keystream

La génération du keystream constitue l'étape finale de l'algorithme ChaCha20.

Après les 20 rounds :

- l'état final est additionné mot à mot à l'état initial ;
- le résultat est converti en 64 octets : le keystream.

Le chiffrement s'effectue par :

texte_chiffré = texte_clair XOR keystream

Le déchiffrement utilise exactement la même opération.

4. Avantages du ChaCha20

ChaCha20 présente de nombreux avantages :

- excellente performance sur CPU classiques,
- Résistant aux fuites d'information matérielles.
- Très sûr et utilisé dans des protocoles comme TLS 1.3 ou OpenSSH.
- Permet de chiffrer des messages très longs sans perte d'information.

5. Analyse de sécurité

ChaCha20 est reconnu comme un algorithme de chiffrement robuste et résistant aux attaques cryptographiques connues à ce jour. Il repose exclusivement sur des opérations simples (addition modulaire, rotation binaire et XOR), ce qui le rend performant et difficile à attaquer.

L'un des éléments fondamentaux de sa sécurité repose sur l'utilisation correcte du nonce. La réutilisation d'un même couple (clé, nonce) pour chiffrer deux messages différents entraîne une vulnérabilité : un attaquant pourrait effectuer un XOR entre les deux textes chiffrés afin d'obtenir des informations sur les messages originaux.

Dans notre projet, une attention a donc été portée à la génération de nonces uniques pour chaque utilisateur.

Cependant, ChaCha20 est un algorithme de chiffrement symétrique et non une fonction de hachage. En pratique, pour le stockage sécurisé des mots de passe, il est recommandé d'utiliser des fonctions de dérivation de clé dédiées comme bcrypt ou Argon2. Le chiffrement symétrique nécessite en effet de conserver une clé secrète ; si cette clé est compromise, l'ensemble des mots de passe peut être déchiffré.

L'utilisation de ChaCha20 n'est pas la meilleure solution pour stocker des mots de passe dans une application réelle.

6. Structure de l'implémentation & Difficultés rencontrées

6.1 - Structure de l'implémentation

Le script est structuré par les fonctions suivantes :

- **remplissage_matrice :**

Cette fonction initialise la matrice de 16 cases avec la base, la clé et le nonce. La matrice est représentée sous forme d'un tableau 1D pour simplifier les opérations. La fonction commence par remplir les 4 premières cases avec la base fixe, puis les 4 cases suivantes avec la clé (en la découpant en segments de 4 octets), et enfin les 4 cases suivantes avec le nonce. Le compteur, qui est parfois utilisé sur une ou deux cases, sera modifié plus tard dans le processus pour chaque bloc de données.

- **rotation :**

La fonction de rotation effectue un décalage circulaire de 32 bits d'un entier donné. Elle prend en entrée un entier x et un nombre n de positions à déplacer les bits vers la gauche. Les bits en trop à gauche sont ramenés à droite pour réaliser une rotation complète, en garantissant que le résultat reste dans les limites de 32 bits (en utilisant un masque 0xffffffff).

- **quarter_round :**

Cette fonction applique l'opération de "quarter round" sur la matrice à partir de quatre indices donnés. Le processus commence par une addition, suivie d'un XOR et d'une rotation, et est répété pour tous les éléments concernés, en respectant un ordre précis d'indices pour mélanger les valeurs de la matrice.

- **generer_keystream :**

Cette fonction génère le keystream utilisé pour chiffrer ou déchiffrer les données. Elle commence par insérer la valeur du compteur dans la matrice. Ensuite, elle applique 8 opérations de "quarter round" sur les indices spécifiés de la matrice, en répétant cette opération 10 fois pour obtenir 20 "rounds" au total. À la fin de ce processus, elle additionne les résultats des opérations avec les valeurs de la matrice de départ, puis les convertit en bytes pour obtenir le keystream final.

- **chiffrer :**

Cette fonction permet de chiffrer le message. Elle divise le message en blocs de 64 octets, génère le keystream correspondant à chaque bloc en utilisant la fonction generer_keystream, et effectue un XOR bit-à-bit entre chaque octet du message et le keystream généré. Le résultat est stocké dans une liste d'octets qui sera ensuite convertie en bytes pour produire le message chiffré.

- **déchiffrer :**

Cette fonction effectue l'opération inverse du chiffrement en appliquant un XOR entre le message chiffré et le keystream généré. Ensuite, les bytes sont convertis en une chaîne de caractères pour retrouver le texte en clair.

6.2 - Difficultés rencontrées

On a d'abord eu des difficultés avec les conversions de types de données. Les clés et nonces sont en bytes, mais l'algorithme nécessite des entiers de 32 bits.

On a donc utilisé `int.from_bytes()` pour transformer les bytes en entiers.

De plus, le message est une chaîne de caractères (str), mais il fallait le convertir en bytes pour qu'il puisse être chiffré, ce qu'on a fait avec `encode('utf-8')`.

Enfin, le compteur étant un entier, il a fallu le manipuler correctement pour l'intégrer dans la matrice à chaque tour.

Au départ, on utilisait une matrice 2D pour les 16 valeurs de la matrice de ChaCha20, mais cela compliquait les manipulations.

On a donc simplifié ça en utilisant un tableau 1D, ce qui a rendu le code plus clair et plus facile à manipuler.

Les principales difficultés ont donc concerné l'implémentation manuelle de l'algorithme. L'utilisation de la bibliothèque pycryptodome s'est révélée plus simple, car les opérations cryptographiques sont déjà implémentées et optimisées. Les difficultés se sont alors limitées à l'intégration correcte dans notre application web.

7. Mise en forme sur notre plateforme WEB

Dans l'objectif d'implémenter ChaCha20 sur notre plateforme WEB, nous avons décidé d'utiliser plusieurs scripts Python utilisant la bibliothèque pycryptodome.

Ces scripts sont les suivants (chemin : /src/Ressources/Py/) :

- **login.py** qui est appelé lors de la connexion, le mot de passe saisi par l'utilisateur est envoyé au script login.py, ainsi que le nonce associé à l'utilisateur.
Le script chiffre le mot de passe saisi par l'utilisateur avec le nonce et la clé secrète.
Le mot de passe chiffré est comparé avec le mot de passe stocké dans la base de données.
Si les deux mots de passe chiffrés correspondent, l'utilisateur est authentifié.
- **crypt.py** qui est appelé lorsqu'un nouveau technicien est ajouté à la base de données, le mot de passe est envoyé au script crypter.py.
Le script génère un mot de passe chiffré et un nonce unique pour cet utilisateur.
- **decrypt.py** qui est appelé lorsque l'administrateur web souhaite modifier les informations d'un technicien (comme son mot de passe) pour déchiffrer le mot de passe chiffré dans la base de données.
- **modifier.py** qui est appelé lorsque l'administrateur web met à jour le mot de passe d'un technicien dans le formulaire de modification.
Ce dernier chiffre le nouveau mot de passe en utilisant le même nonce que celui stocké dans la base de données.
Le mot de passe chiffré et le nonce mis à jour sont ensuite renvoyés au script PHP, qui met à jour la base de données avec ces nouvelles valeurs.

8. Limites et perspectives

Bien que l'implémentation réalisée respecte les spécifications de la RFC 8439, certaines améliorations pourraient être envisagées.

Tout d'abord, des tests unitaires automatisés pourraient être ajoutés afin de vérifier la conformité des résultats. Cela garantit une validation du bon fonctionnement de l'algorithme.

Ensuite, une comparaison des performances entre l'implémentation manuelle et celle utilisant une bibliothèque spécialisée permettrait d'analyser l'écart d'optimisation.

Enfin, une extension du projet serait l'étude et l'implémentation de ChaCha20-Poly1305, qui combine chiffrement et authentification, proposant un mécanisme complet de sécurité garantissant à la fois confidentialité et intégrité des données.

8.1 - Comparaison implémentation manuelle vs bibliothèque

L'implémentation manuelle permet de comprendre le fonctionnement interne de ChaCha20, mais elle présente un risque d'erreur plus élevé.

L'utilisation d'une bibliothèque spécialisée comme pycryptodome offre :

- une meilleure optimisation des performances ;
- une réduction des risques d'erreur ;
- une conformité garantie aux standards cryptographiques.

Cependant, elle ne permet pas d'observer directement les mécanismes internes de l'algorithme.

Conclusion

Ce projet nous a permis de comprendre en profondeur le fonctionnement d'un algorithme de chiffrement par flot moderne. L'implémentation complète de ChaCha20 a nécessité une maîtrise précise des opérations arithmétiques sur 32 bits, de la gestion des conversions de types ainsi qu'une organisation du code.

Nous avons pris conscience que la moindre erreur dans la gestion du nonce et du compteur peut compromettre la confidentialité des données.

L'intégration de ChaCha20 dans notre plateforme web nous a également permis de relier théorie cryptographique et application concrète, renforçant notre compréhension des enjeux de sécurité dans le développement logiciel.

Nous avons particulièrement pris conscience de l'importance du nonce et du compteur dans la sécurité globale du système. La moindre erreur dans leur gestion peut compromettre la confidentialité des données.

9. Sources/Bibliographies

Nir, Y., & Langley, A. (2018). RFC 8439: ChaCha20 and Poly1305 for IETF Protocols. IETF.

Daniel J. Bernstein – ChaCha, a variant of Salsa20.

Wikipedia – Article ChaCha20.

<https://www.bortzmeyer.org/8439.html>

<https://www.rfc-editor.org/rfc/pdf/rfc8439.txt.pdf>

<https://muens.io/chacha20/>

https://en.wikipedia.org/wiki/Salsa20#ChaCha_variant

<https://blog.cloudflare.com/it-takes-two-to-chacha-poly/>

[Vidéo explicative sur ChaCha20](#)